MYSQL TRIGGERS

Practical Examples and Student Exercises using MySQL Workbench

What is a Trigger in MySQL?

A Trigger is a database object that automatically executes when a specified event occurs on a table.

Note: You don't call a trigger manually. MySQL automatically executes it when INSERT, UPDATE, or DELETE happens.

Why do we use Triggers?

Triggers are mainly used for:

- **Auditing** – record who/what was changed
- **Validation** – prevent invalid data
- **Automatic calculations**
- **Maintaining related/summary data**
- **Logging INSERT, UPDATE and DELETE operations**
- **Enforcing business rules**

Types of MySQL Triggers

MySQL triggers are classified based on **when** they execute and **what event** activates them.

Timing

1. BEFORE
2. AFTER

Events

1. INSERT
2. UPDATE
3. DELETE

- **First Trigger Example – AFTER INSERT**

Create an employees table:

```
CREATE TABLE employee(
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary DECIMAL(10,2)
);
CREATE TABLE employee_audit(
```

Create an audit table:

```
CREATE TABLE employee_audit(
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    emp_name VARCHAR(50),
    action_type VARCHAR(20),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Now create the trigger:

```
DELIMITER //
CREATE TRIGGER after_employee_insert
AFTER INSERT ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit
    (emp_id, emp_name, action_type)
    VALUES
    (NEW.emp_id, NEW.emp_name, 'INSERT');
END //
```

Now insert an employee:

```
    END //
    DELIMITER ;
    INSERT INTO employee
    VALUES (106, 'Arjun', 1, 52000);
```

➢ You **didn't call the trigger**. It automatically executes.

```
SELECT * FROM employee_audit;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

audit_id	emp_id	emp_name	action_type	action_time
1	106	Arjun	INSERT	2026-08-10 11:23:52
NULL	NULL	NULL	NULL	NULL

- **Second Trigger Example – AFTER DELETE**

Suppose Employee deleted:

```
DELETE FROM employee
WHERE emp_id = 106;
```

Output

Action Output

#	Time	Action	Message
73	11:23:27	CREATE TABLE employee(emp_id INT PRIMARY KEY, emp_name VARCHA...	0 row(s) affected
74	11:23:36	CREATE TABLE employee_audit(audit_id INT AUTO_INCREMENT PRIMARY ...	0 row(s) affected
75	11:23:42	CREATE TRIGGER after_employee_insert AFTER INSERT ON employee FOR EA...	0 row(s) affected
76	11:23:52	INSERT INTO employee VALUES (106, 'Arjun', 1, 52000)	1 row(s) affected
77	11:24:20	SELECT * FROM employee_audit LIMIT 0, 1000	1 row(s) returned
78	11:25:17	DELETE FROM employee WHERE emp_id = 106	1 row(s) affected

We can automatically record that deletion.

```
CREATE TRIGGER after_employee_delete
AFTER DELETE ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit
    (emp_id, emp_name, action_type)
    VALUES
    (OLD.emp_id, OLD.emp_name, 'DELETE');
END //
DELIMITER ;
```

lt Grid | Filter Rows: | Edit:

emp_id	emp_name	dept_id	salary
ULL	NULL	NULL	NULL

- **Third Trigger Example--AFTER UPDATE Trigger:**

We can record the update.

```
UPDATE employee
SET salary = 60000
WHERE emp_id = 101;

drop trigger if exists after_employee_update;
DELIMITER //
CREATE TRIGGER after_employee_update
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
```

```
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit
    (emp_id, emp_name, action_type)
    VALUES
    (NEW.emp_id, NEW.emp_name, 'UPDATE');
END //
DELIMITER ;
select * from employee;
```

Result Grid | Filter Rows: | Edit: | Export/I

emp_id	emp_name	dept_id	salary
101	John	1	60000.00
NULL	NULL	NULL	NULL

- **Fourth Trigger Example--BEFORE INSERT Trigger:**

Employee salary cannot be less than ₹20,000.

```

DELIMITER //
CREATE TRIGGER check_salary
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 20000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be less than 20000';
    END IF;
END //
DELIMITER ;
INSERT INTO employee VALUES (107, 'Ravi', 1, 15000);
INSERT INTO employees VALUES (108, 'Meena', 1, 30000);
```

Now, run this Query

INSERT INTO employees

VALUES (107, 'Ravi', 1, 15000);

```
295 •  INSERT INTO employee VALUES (107, 'Ravi', 1, 15000);
296 •  INSERT INTO employee VALUES (108, 'Meena', 1, 30000);
```

Output

Action Output

#	Time	Action	Message
88	11:32:17	CREATE TRIGGER after_employee_update AFTER UPDATE ON employee FOR ...	Error Code: 1359. Trigger already exists
89	11:33:35	drop trigger if exists after_employee_update	0 row(s) affected
90	11:33:40	CREATE TRIGGER after_employee_update AFTER UPDATE ON employee FOR ...	0 row(s) affected
91	11:33:45	select * from employee LIMIT 0, 1000	1 row(s) returned
92	11:37:21	CREATE TRIGGER check_salary BEFORE INSERT ON employee FOR EACH RO...	0 row(s) affected
93	11:38:16	INSERT INTO employee VALUES (107, 'Ravi', 1, 15000)	Error Code: 1644. Salary cannot be less than 20000

MySQL will reject it and display:

Salary cannot be less than 20000

But, If you run this Query it will succeed as salary is greater than 20000

```
295 •  INSERT INTO employee VALUES (107, 'Ravi', 1, 15000);
296 •  INSERT INTO employee VALUES (108, 'Meena', 1, 30000);
```

Output

Action Output

#	Time	Action	Message
89	11:33:35	drop trigger if exists after_employee_update	0 row(s) affected
90	11:33:40	CREATE TRIGGER after_employee_update AFTER UPDATE ON employee FOR ...	0 row(s) affected
91	11:33:45	select * from employee LIMIT 0, 1000	1 row(s) returned
92	11:37:21	CREATE TRIGGER check_salary BEFORE INSERT ON employee FOR EACH RO...	0 row(s) affected
93	11:38:16	INSERT INTO employee VALUES (107, 'Ravi', 1, 15000)	Error Code: 1644. Salary cannot be less than 20000
94	11:39:51	INSERT INTO employee VALUES (108, 'Meena', 1, 30000)	1 row(s) affected

- ## Fifth Trigger Example--BEFORE UPDATE Trigger:

Suppose the company does not allow salary reduction.

```
297
298 •  drop trigger if exists prevent_salary_reduction;
299    DELIMITER //
300 •  CREATE TRIGGER prevent_salary_reduction
301    BEFORE UPDATE ON employee
302    FOR EACH ROW
303    BEGIN
304        IF NEW.salary < OLD.salary THEN
305            SIGNAL SQLSTATE '45000'
306            SET MESSAGE_TEXT = 'Salary reduction is not allowed';
307        END IF;
308    END //
309    DELIMITER ;
```

Now, run this Update Query

```
UPDATE employee
SET salary = 40000
WHERE emp_id = 101;
```

Context Help

Output

Action Output

#	Time	Action	Message
95	11:41:03	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employees F...	0 row(s) affected
96	11:41:12	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	Error Code: 1359. Trigger already exists
97	11:41:42	drop trigger if exists prevent_salary_reduction	0 row(s) affected
98	11:41:46	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	0 row(s) affected
99	11:42:04	UPDATE employees SET salary = 40000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
100	11:42:46	UPDATE employee SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed

If the old salary was ₹50,000, the trigger rejects the update.

But, if you run the below update query, it succeeds as you are not reducing salary

```
UPDATE employees
SET salary = 60000
WHERE emp_id = 101;
```

Output

Action Output

#	Time	Action	Message
96	11:41:12	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	Error Code: 1359. Trigger already exists
97	11:41:42	drop trigger if exists prevent_salary_reduction	0 row(s) affected
98	11:41:46	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employee FO...	0 row(s) affected
99	11:42:04	UPDATE employees SET salary = 40000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
100	11:42:46	UPDATE employee SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed
101	11:44:23	UPDATE employees SET salary = 60000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
UPDATE employee
SET salary = 70000
WHERE emp_id = 101;
select * from employee;
```

Result Grid | Filter Rows: | Edit:

emp_id	emp_name	dept_id	salary
101	John	1	70000.00
108	Meena	1	30000.00
NULL	NULL	NULL	NULL

Here, after salary update

OLD.salary → 60000

NEW.salary → 70000